

Insurance Portal Workflow

Eduardo Bezerra
Francisco Tavares
Rafael Reis

Orientadores: Artur Ferreira
Pedro Miguens Matutino

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Julho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Insurance Portal Workflow

46362 Eduardo Franco Bezerra

51618 Francisco Duarte Tavares

51870 Rafael Henrique Souza Reis

Orientadores: Artur Jorge Ferreira

Pedro Miguel Florindo Miguens Matutino

Relatório do projeto realizado no âmbito de Projeto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Semestre de Verão de 2025/2026

Julho de 2026

Resumo

A CAPT Consulting, é quem requeriu a prova de conceito aqui apresentada, é uma empresa especializada na averiguação de sinistros no setor segurador. Assim, considera-se que um processo de averiguação inicia-se com a receção de um pedido submetido por uma entidade cliente da CAPT, sendo que estes pedidos são recebidos de forma não uniforme, tanto ao nível do canal de comunicação como do formato de submissão.

Com a gestão atual, a CAPT enfrenta dificuldades na obtenção de uma visão global e centralizada dos processos de averiguação em curso, bem como na uniformização e normalização dos procedimentos operacionais associados. Esta limitação decorre, em grande medida, da dispersão da informação e da inexistência de uma plataforma integrada de suporte à gestão destes processos.

Com o objetivo de solucionar estas limitações, foi concebida e desenvolvida uma plataforma *Web*, designada *IPW* (*Insurance Portal Workflow*), orientada para a centralização da gestão de averiguações, a normalização de fluxos de trabalho e a disponibilização de uma visão operacional abrangente sobre toda a atividade de averiguação da organização.

Palavras-chave: *workflow*; averiguação de sinistros; seguros; plataforma web; gestão de processos; centralização de informação.

Abstract

CAPT Consulting is the entity that requested the proof of concept presented here, is a company specialized in the investigation of insurance claims within the insurance sector. Thus, an investigation process is considered to begin with the receipt of a request submitted by a CAPT client entity, and these requests are received in a non-uniform manner, both in terms of the communication channel and the submission format.

Under the current management, CAPT faces difficulties in obtaining a global and centralized view of ongoing investigation processes, as well as in standardizing and normalizing the associated operational procedures. This limitation largely stems from the dispersion of information and the absence of an integrated platform to support the management of these processes.

In order to address these limitations, a web-based platform, named IPW (Insurance Portal Workflow), was designed and developed. The system is aimed at centralizing the management of investigations, standardizing workflow processes, and providing a comprehensive operational overview of the organization's investigative activities.

Keywords: *workflow*; claims investigation; insurance; web platform; process management; information centralization.

Agradecimentos

Aproveito este espaço para agradecer a todo o corpo docente do Instituto Superior de Engenharia de Lisboa por ter feito parte da minha formação académica e por ter contribuído para o meu crescimento pessoal e profissional.

Agradeço especialmente aos nossos orientadores, Artur Ferreira e Pedro Matutino, pelo apoio, pela orientação e pela disponibilidade ao longo do desenvolvimento deste projeto.

Por último, agradeço à minha mãe pelo constante incentivo e apoio incondicional. Muito obrigado.

Julho 2022, Eduardo Bezerra

Francisco

Rafael

Utilização de Inteligência Artificial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa como apoio complementar ao desenvolvimento do projeto. A utilização destas ferramentas incidiu principalmente em quatro áreas: apoio à redação e revisão textual do relatório, auxílio na concepção do *layout* e da organização visual da plataforma *Web*, suporte ao desenvolvimento de testes unitários e realização de traduções entre Português e Inglês. Para auxílio na realização das tarefas acima referidas, foi utilizado maioritariamente o GitHub Copilot, com o modelo Claude Sonnet 4.6.

No contexto da documentação, as ferramentas foram utilizadas para auxiliar a reformulação de texto, melhoria da clareza técnica, correção gramatical e tradução de conteúdos entre Português e Inglês. Relativamente à interface *Web*, foram utilizadas para obtenção de sugestões de organização visual, estruturação de componentes e melhoria da experiência de utilização. No desenvolvimento de *software*, foram também utilizadas como apoio na criação e revisão de testes unitários.

A utilização destas ferramentas teve um carácter exclusivamente assistivo, tendo todas as decisões técnicas, validações, implementações e adaptações finais sido realizadas pelos autores do projeto.

Acrónimos e Siglas

API *Application Programming Interface*

CSS *Cascading Style Sheets*

HTTP *HyperText Transfer Protocol*

IPW *Insurance Portal Workflow*

JSON *JavaScript Object Notation*

JWT *JSON Web Token*

RBAC *Role-Based Access Control*

REST *Representational State Transfer*

SQL *Structured Query Language*

Índice

1	Introdução	1
1.1	Objetivos	3
1.2	Estado da Arte	4
1.3	Organização do documento	4
2	Solução Proposta	5
2.1	Arquitetura	5
2.2	Modelo de dados	7
2.2.1	Autenticação	7
2.2.2	Gestão de processos	9
2.2.3	Modelo completo	15
2.2.4	Implementação	16
2.3	Backend e segurança	18
2.4	Servidor Frontend	19
2.4.1	Bibliotecas Utilizadas	20
3	Demonstração da Aplicação	21
3.1	Autenticação e seleção de papel	21
3.2	Listagem e acompanhamento de processos	22
3.3	Criação de processos	23
3.4	Detalhe do processo	23
3.5	Gestão de utilizadores	24
3.6	Perfil e histórico de atividade	25
4	Conclusão e Trabalho Futuro	27
4.1	Conclusão	27
4.2	Trabalho futuro	27
	Referências	29

Lista de Figuras

1.1	Perfis de utilizadores da na plataforma IPW.	2
2.1	Arquitetura do sistema.	6
2.2	Entidades de autenticação.	8
2.3	Diagrama do <i>workflow</i> e ciclo de vida dos estados de um processo.	9
2.4	Entidade processo e características.	11
2.5	Entidades de documentação, provas e notas do processo.	12
2.6	Mapeamento relacional incluindo processos, utilizadores, áreas de especialidade e atividades.	13
2.7	Modelo entidade-associação completo (sem atributos).	15
2.8	Estrutura de pastas da componente <i>frontend</i>	20
3.1	Página de autenticação da aplicação.	21
3.2	Seleção do papel ativo na sessão do utilizador.	22
3.3	Exemplo de listagem de processos disponível para papéis envolvidos no acompanhamento.	22
3.4	Formulário de criação de um novo processo pelo triador.	23
3.5	Página de detalhe de um processo de averiguação.	24
3.6	Interface de gestão de utilizadores disponível ao administrador.	24
3.7	Página de perfil e consulta de atividade recente do utilizador.	25

Capítulo 1

Introdução

Um processo de averiguação inicia-se com a receção, por parte da CAPT, de um pedido submetido por um dos seus clientes, habitualmente seguradoras. Estes pedidos contêm informação preliminar relativa ao sinistro, nomeadamente a localização da ocorrência, os intervenientes envolvidos, a data deste, entre outros elementos relevantes para a abertura do processo de averiguação.

Atualmente, os pedidos são recebidos de forma não uniforme, o que introduz dificuldades operacionais no tratamento e acompanhamento dos processos, o que obriga os colaboradores da CAPT a recorrer a procedimentos pouco padronizados. A inexistência de uma plataforma centralizada limita ainda a capacidade da gestão em obter uma visão global da operação e em tomar decisões informadas, o que pode originar situações de distribuição desequilibrada de carga de trabalho e incumprimento de prazos.

Após a receção de um pedido de averiguação, é necessário proceder à sua análise e atribuição de um averiguador e de um supervisor. O averiguador desloca-se ao local da ocorrência, realiza as diligências necessárias, recolhendo evidências relevantes e elabora um relatório técnico que contém toda a informação apurada durante o processo de investigação. Posteriormente, esse relatório é submetido para validação pelo supervisor, responsável por garantir que o conteúdo produzido cumpre os padrões de qualidade e conformidade definidos pela CAPT antes da sua entrega ao cliente final.

Com a implementação da plataforma IPW (*Insurance Portal Workflow*), pretende-se criar um ambiente de trabalho mais estruturado, centralizado e normalizado, permitindo uma definição clara das responsabilidades de cada interveniente e promovendo uma maior rastreabilidade dos processos de averiguação.

Para suportar esta organização operacional, foram definidos cinco perfis distintos de utilizador:

- **Triador** — responsável pela receção dos pedidos de averiguação, criação de processos e pela sua atribuição aos averiguadores e supervisores;
- **Averiguador** — responsável pela realização das investigações no terreno e pela ela-

boração do respetivo relatório técnico;

- **Supervisor** — responsável pela validação dos relatórios técnicos produzidos pelos averiguadores de uma determinada área de especialização, como, por exemplo, sinistros automóveis ou acidentes de trabalho;
- **Gestor** — responsável pela validação final dos relatórios aprovados pelos supervisores, assegurando um segundo nível de controlo e supervisão sobre as diferentes áreas operacionais da organização;
- **Administrador** — responsável pela gestão da plataforma IPW, incluindo a criação de utilizadores, atribuição de perfis e gestão de permissões de acesso.

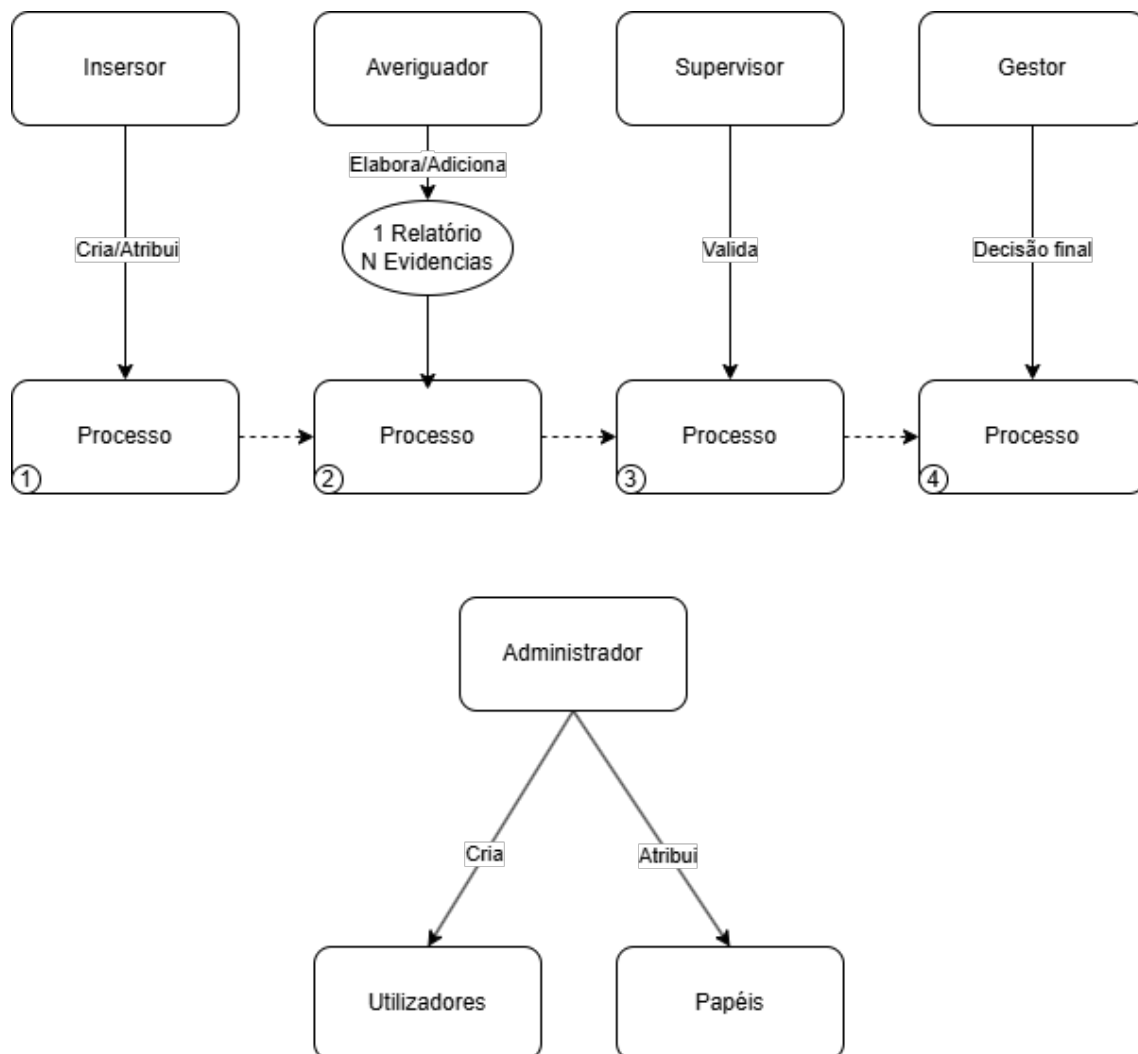


Figura 1.1: Perfis de utilizadores da na plataforma IPW.

1.1 Objetivos

O presente projeto tem como principal objetivo o desenvolvimento da plataforma *Web, IPW (Insurance Portal Workflow)*, para suporte à gestão de processos de averiguação de sinistros da CAPT Consulting. A solução pretende melhorar a organização operacional da empresa, promovendo maior uniformização de procedimentos, rastreabilidade da informação e capacidade de monitorização da atividade.

De forma mais específica, definiram-se os seguintes objetivos:

- Centralizar a informação relativa aos processos de averiguação numa única plataforma integrada;
- Estruturar o processo de atribuição e acompanhamento de averiguações entre os diferentes intervenientes da organização;
- Garantir o registo e rastreabilidade das interações realizadas ao longo do ciclo de vida de um processo;
- Disponibilizar mecanismos de gestão documental associados aos processos de averiguação;
- Fornecer indicadores operacionais e ferramentas de monitorização que suportem a supervisão e a tomada de decisão.

1.2 Estado da Arte

Existem diversas aplicações de *workflow* disponíveis no mercado:

- *E-clic* [1]: É uma plataforma empresarial de gestão documental e automação de *workflows*, focada no controlo, organização e rastreabilidade de documentos e processos internos.
- *DocuWare* [2]: Plataforma cloud de gestão documental e automação de *workflows* empresariais, utilizada para digitalizar processos, arquivar documentos e controlar aprovações internas.
- *Microsoft SharePoint* [3]: Plataforma colaborativa da *Microsoft* focada na gestão de documentos, intranets corporativas e partilha de informação entre equipas. É amplamente utilizada em ambientes empresariais integrados com o ecossistema *Microsoft 365*.

Apesar da existência deste tipo de aplicação ser muito ampla no mercado, este projeto focou-se em desenvolver um portal específico para a CAPT Consulting. A plataforma IPW foi concebida para responder às necessidades específicas da empresa acima referida, tendo em consideração as particularidades dos processos de averiguação de sinistros e a estrutura organizacional da empresa, é personalizada para o contexto da CAPT, o que promove uma maior facilidade a adoção por parte dos seus colaboradores.

1.3 Organização do documento

O presente relatório encontra-se estruturado em quatro capítulos.

No Capítulo 1 procede-se ao enquadramento do problema, à apresentação dos objetivos do projeto e à contextualização da plataforma IPW.

O Capítulo 2 é dedicado à descrição da solução proposta, incluindo a arquitetura do sistema, o modelo de dados e os principais aspetos de implementação das componentes *backend* e *frontend*.

No Capítulo 3 é apresentada a demonstração da aplicação, com incidência nos ecrãs desenvolvidos.

Finalmente, o Capítulo 4 reúne as conclusões do trabalho realizado e identifica as principais perspetivas de desenvolvimento futuro.

Capítulo 2

Solução Proposta

Este capítulo apresenta detalhadamente a arquitetura, o modelo de dados e os detalhes de implementação da solução tecnológica desenvolvida. O conteúdo encontra-se estruturado em quatro secções principais: a Secção 2.1 descreve a arquitetura geral do sistema baseada num modelo *client-server* em camadas; a Secção 2.2 detalha o modelo de dados conceptual (Entidade-Associação) e as regras de negócio automatizadas na base de dados; a Secção 2.3 aborda a componente de *backend*, focando-se no desenvolvimento com *Spring Boot* e nos mecanismos de segurança e controlo de acessos (*RBAC*) com *Spring Security*; por fim, a Secção 2.4 expõe a estrutura do servidor *frontend* em *React*, os seus componentes de proteção de rotas e as bibliotecas externas integradas.

2.1 Arquitetura

Na Figura 2.1 representa-se a arquitetura geral da aplicação desenvolvida no âmbito deste projeto. A solução foi concebida segundo um modelo de arquitetura *client-server*, estruturado em diferentes camadas lógicas, de forma a garantir uma clara separação de responsabilidades e uma melhor organização do sistema.

De um modo geral, a arquitetura baseia-se na interação entre três grandes blocos: o *frontend*, o *backend* e a base de dados. Esta organização permite isolar as preocupações de cada componente, facilitando a evolução independente das diferentes partes do sistema.

A comunicação entre os vários componentes é realizada através de uma *API* baseada em serviços *REST*, assegurando um modelo de interação padronizado e independente da tecnologia utilizada no *frontend*. Adicionalmente, foram adotados mecanismos de segurança para controlo de acesso e autenticação de utilizadores, garantindo a proteção dos recursos da aplicação.

Esta abordagem arquitetural favorece a modularidade, escalabilidade e manutenibilidade do sistema, permitindo ainda uma melhor adaptação a futuras extensões ou alterações funcionais.

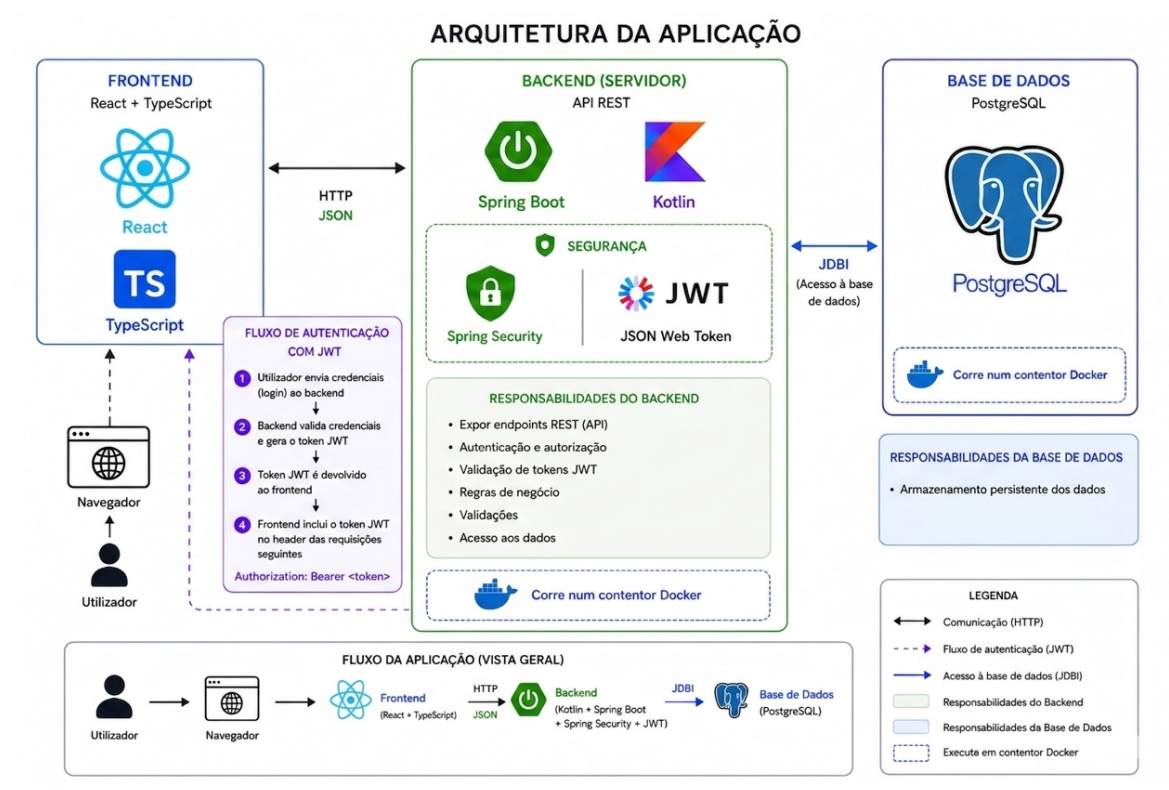


Figura 2.1: Arquitetura do sistema.

2.2 Modelo de dados

2.2.1 Autenticação

No que respeita ao conjunto de entidades responsável pela autenticação e autorização de utilizadores, existem cinco entidades principais: *User*, *AccessToken*, *RefreshToken*, *LoginToken* e *Role*. O modelo conceptual e a estrutura destas componentes encontram-se ilustrados na Figura 2.2.

Relativamente à entidade *User*, esta é composta por um identificador único (*id*), um nome (*name*), uma *string* correspondente ao *hash* da palavra-passe do utilizador (*password validation*) e um indicador responsável por informar se o utilizador se encontra ativo (*isActive*).

No que concerne à entidade *AccessToken*, esta é composta por um *token*, uma data de expiração (*expiration date*) e um papel ativo associado (*active role*). O propósito fundamental do *AccessToken* consiste em autenticar os pedidos do utilizador e, simultaneamente, identificar o papel específico (*Role*) que este vai desempenhar durante aquela determinada sessão.

A entidade *RefreshToken* apresenta uma estrutura idêntica à do *AccessToken*. No entanto, o seu propósito específico consiste em permitir a obtenção de um novo *AccessToken* assim que este expire, garantindo a continuidade da sessão sem obrigar o utilizador a realizar um novo processo de *login*.

Relativamente ao *LoginToken*, este apresenta uma estrutura semelhante à das entidades anteriores, distinguindo-se pelo facto de não possuir qualquer papel associado e de manter o atributo de data de criação (*creation date*). Este atributo é fundamental para conseguir guardar o registo da última sessão do utilizador, permitindo à aplicação saber a última vez que este esteve *online*. Adicionalmente, o *LoginToken* serve exclusivamente para autenticar inicialmente o utilizador no sistema e permitir a verificação de todos os papéis (*Roles*) que este tem disponíveis para assumir.

A entidade *Role* é constituída apenas por um nome (*name*), o qual está estritamente limitado a cinco valores possíveis: *Admin*, *Triator*, *Investigator*, *Supervisor* e *Manager*. Esta limitação a um conjunto fixo de valores funciona como uma restrição de domínio (semelhante a um tipo enumerado), garantindo a integridade dos dados ao impedir a criação de papéis arbitrários e assegurando que o controlo de acessos no sistema se restringe apenas às funções previstas pelas regras de negócio da aplicação.

No que respeita às relações entre estas entidades, um utilizador pode estar associado a diferentes *LoginToken*, *AccessToken* e *RefreshToken*, permitindo a autenticação e a manutenção da sessão do mesmo nas diferentes etapas da aplicação.

Relativamente à relação entre as entidades de *token* (*AccessToken* e *RefreshToken*) e a entidade *Role*, esta existe devido à necessidade de o *token* de acesso e de renovação transportarem informação acerca do papel desempenhado pelo utilizador durante uma determinada sessão.

No que concerne à relação entre *User* e *Role*, um utilizador pode possuir múltiplos papéis e, simultaneamente, o mesmo papel pode estar associado a diferentes utilizadores, caracterizando assim uma relação de *N*-para-*N*.

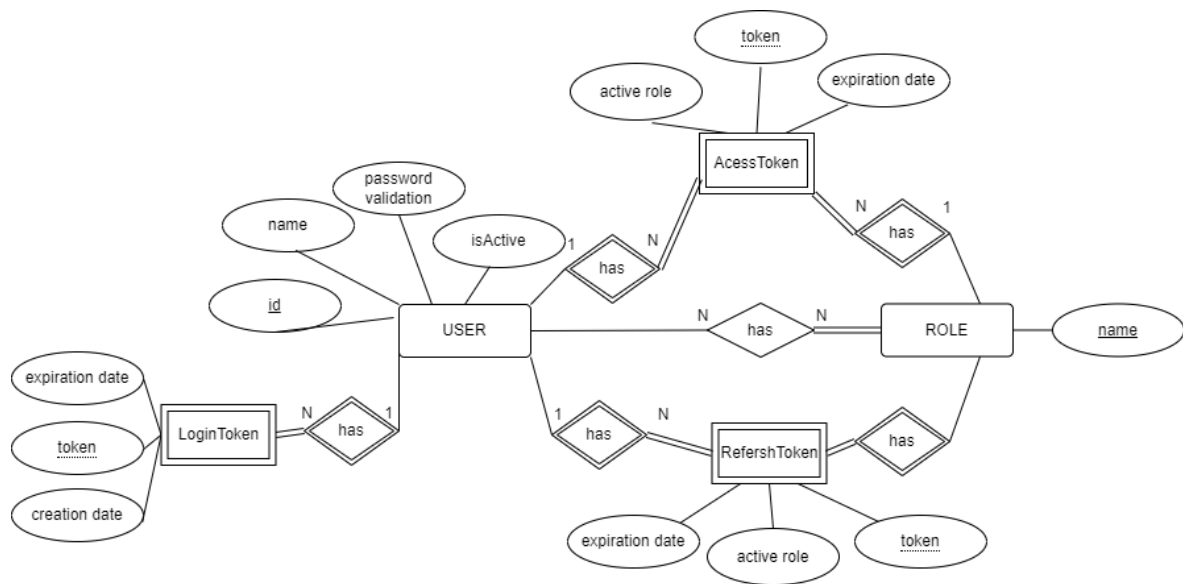


Figura 2.2: Entidades de autenticação.

2.2.2 Gestão de processos

No que respeita ao conjunto de entidades associado à gestão de processos, destacam-se as entidades *Process*, *Insurance*, *State* e *Location*. Estas entidades são responsáveis por representar a informação principal associada aos processos geridos pela aplicação, bem como o seu enquadramento, estado e classificação.

Relativamente à entidade *Process*, esta constitui a entidade central do domínio da aplicação. É composta por um identificador único (*id*), um nome (*name*), um nível de prioridade (*priority*), uma data de inserção no sistema (*creation date*), uma data prevista de conclusão (*due date*).

No que concerne à entidade *Insurance*, esta representa a seguradora associada ao processo. É constituída por um identificador único (*id*) e por um nome (*name*).

Recorre-se à entidade *State* para representar o estado atual de um processo ao longo do seu ciclo de vida, sendo esta composta por um identificador único (*id*) e um nome (*name*). Por forma a garantir o cumprimento estrito das regras de negócio, os estados possíveis encontram-se rigidamente delimitados a um conjunto predefinido de valores, cuja sequência operacional e transições lógicas estão ilustradas na Figura 2.3.

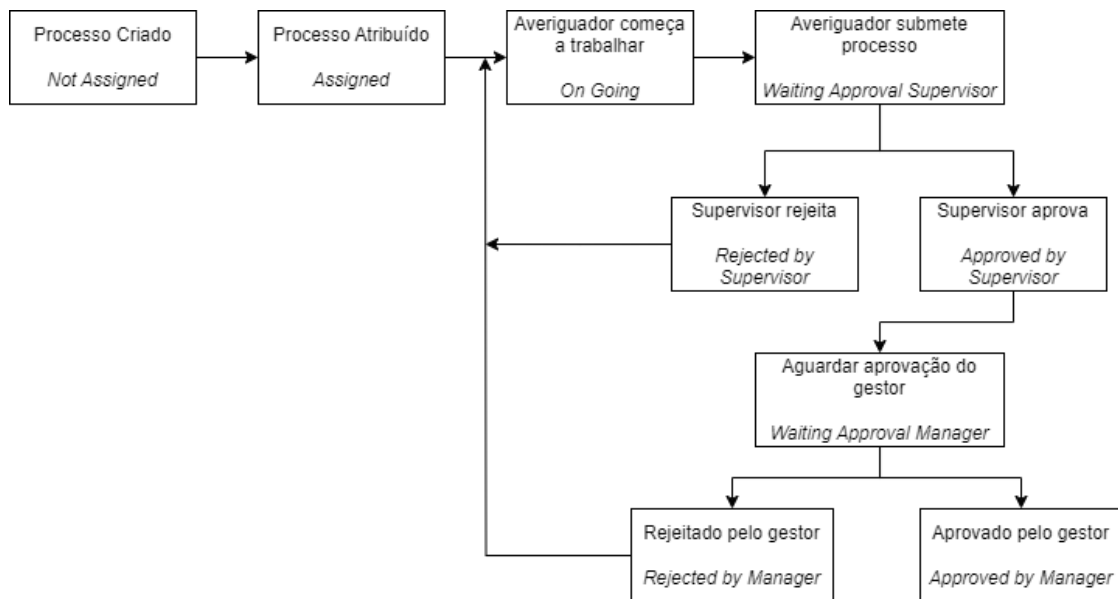


Figura 2.3: Diagrama do *workflow* e ciclo de vida dos estados de um processo.

O fluxo inicia-se obrigatoriamente com a criação do processo no estado *Not Assigned*. Assim que a equipa responsável é definida (averiguador e supervisor), o processo evolui para *Assigned*. O ciclo operacional propriamente dito começa quando o averiguador inicia a recolha de provas e criação do relatório, alterando o estado para *On Going*. Após a conclusão da recolha de dados e preenchimento dos relatórios, o averiguador submete o processo, o qual transita automaticamente para *Waiting Approval Supervisor*.

A partir deste ponto crítico de validação, o *workflow* ramifica-se de acordo com as decisões das hierarquias de supervisão e gestão:

- **Fluxo de Supervisão:** O supervisor avalia o trabalho submetido. Caso identifique desconformidades, o processo passa para *Rejected by Supervisor*, sendo forçado a retroceder diretamente para o estado *On Going* de modo a permitir que o averiguador efetue as devidas correções. Se o parecer for positivo, o estado passa a *Approved by Supervisor*, encaminhando o processo de imediato para a fila de verificação seguinte sob o estado *Waiting Approval Manager*.
- **Fluxo de Gestão:** O gestor (*Manager*) assume a palavra final sobre o processo. Caso detete a necessidade de alterações profundas ou revisão, aplica o estado *Rejected by Manager*, o que aciona um mecanismo de retorno ao estado de execução activa (*On Going*). Havendo total conformidade com os requisitos institucionais, o gestor concede a validação definitiva, culminando no estado terminal de sucesso *Approved by Manager*.

Adicionalmente, importa salvaguardar que, independentemente da fase em que se encontre na linha temporal descrita, qualquer processo pode ser cancelado a qualquer momento por decisão exclusiva do *Manager*, assumindo nessa circunstância o estado de encerramento *Canceled*.

Por fim, a entidade *Location* representa a localização associada ao sinistro. Esta é composta por um identificador único (*id*), o distrito (*district*), o concelho (*county*), a rua (*street*), a latitude (*latitude*) e a longitude (*longitude*).

No que respeita às relações entre estas entidades, um processo encontra-se associado a uma seguradora, podendo a mesma seguradora estar associada a múltiplos processos, caracterizando assim uma relação de 1-para-N entre *Insurance* e *Process*.

Relativamente à relação entre *Process* e *State*, um processo pode assumir diferentes estados ao longo do seu ciclo de vida, sendo possível que o mesmo estado esteja associado a diferentes processos. A cada relação entre estado e processo são armazenados a data de início (*start date*) e a data de término (*end date*), sendo que, caso seja o estado atual, este valor será nulo (*null*). Esta relação permite representar a evolução e acompanhamento do processo dentro do sistema.

Por último, no que concerne à relação entre *Process* e *Location*, cada processo de sinistro encontra-se associado a uma localização específica, enquanto uma localização pode registar várias ocorrências.

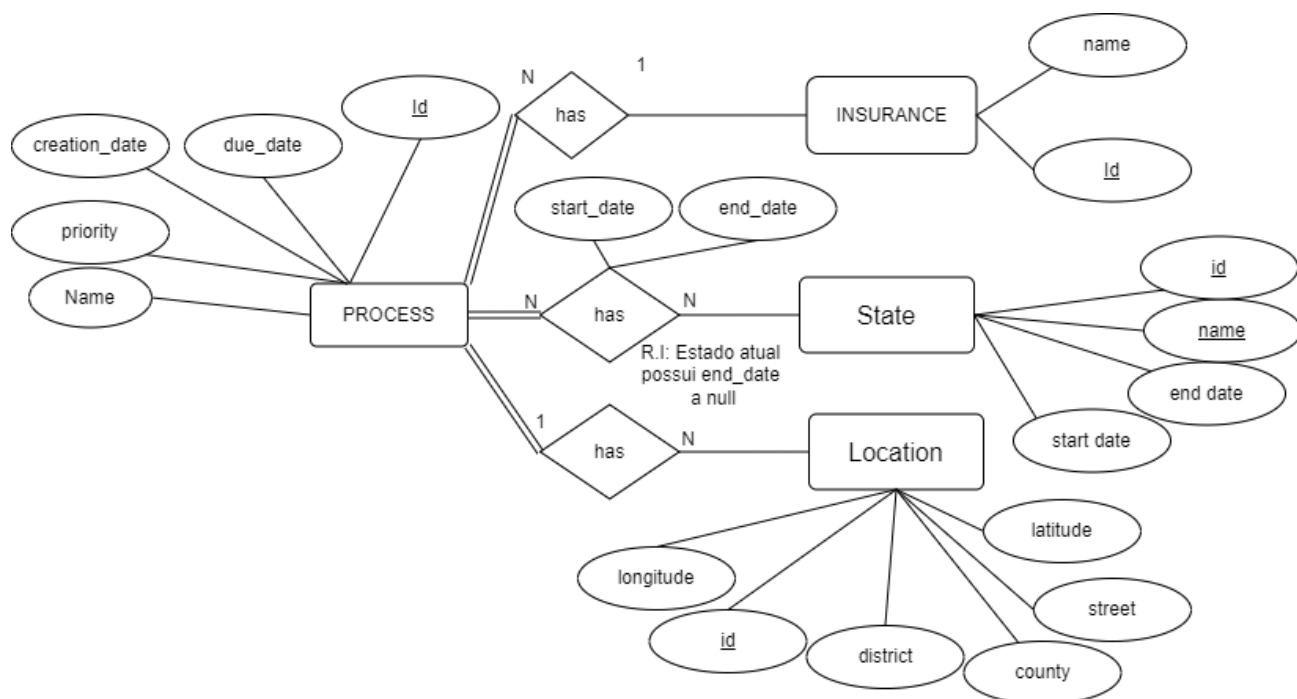


Figura 2.4: Entidade processo e características.

Documentação e Provas do Processo

No que respeita ao conjunto de entidades associado à documentação, relatórios e suporte probatório dos sinistros, destacam-se as entidades *Report*, *Prove* e *Note*. Estas componentes são fundamentais para assegurar a recolha de evidências, o registo de observações contínuas e a formalização das conclusões obtidas ao longo da averiguação de um processo. A estrutura e o mapeamento relacional deste módulo encontram-se ilustrados na Figura 2.5.

Relativamente à entidade *Report*, esta representa o relatório final detalhado pelo averiguador. É composta por um identificador único (*id*), o conteúdo do relatório (*content*) e uma data de criação (*creation date*) e, para caso seja necessário atualizar após a resposta dos superiores, uma data de modificação (*updated at*). O propósito desta entidade é consolidar formalmente todas as conclusões para posterior apreciação por parte dos supervisores e gestores.

No que concerne à entidade *Prove*, esta destina-se ao armazenamento de evidências e elementos de prova recolhidos em sede de averiguação (tais como fotografias e vídeos). É constituída por um identificador único (*id*), uma designação ou nome do ficheiro (*file name*), o tipo de ficheiro (*file type*), o *url* que aponta para o recurso (*file url*) e a respetiva data de carregamento (*creation date*).

Por fim, a entidade *Note* funciona como um histórico de anotações internas. Permite que qualquer utilizador associado a um processo deixe um comentário, observações pontuais ou notas de esclarecimento. Esta entidade é composta por um identificador único (*id*), o

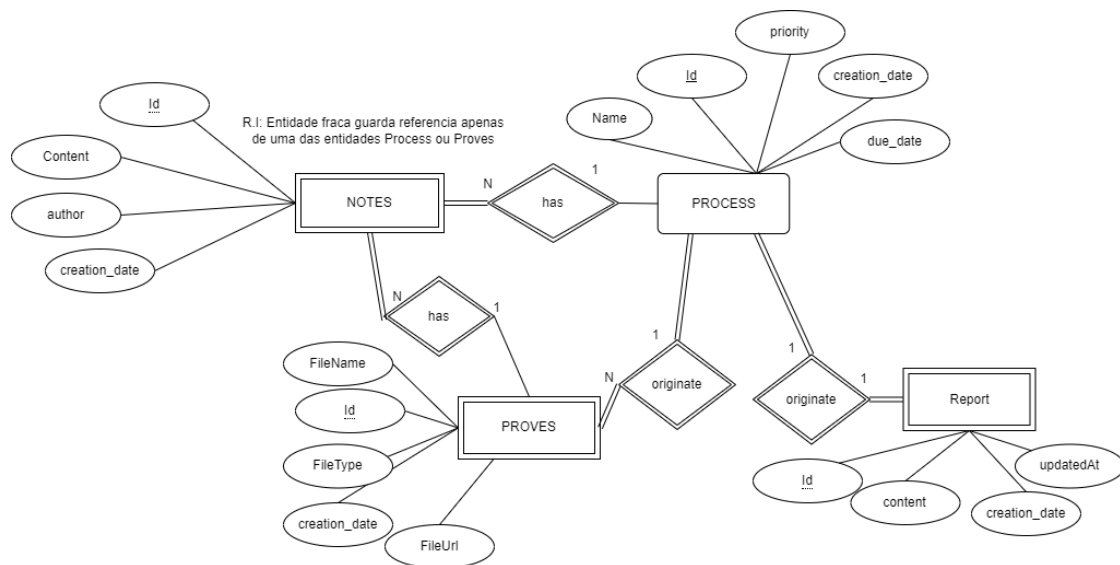


Figura 2.5: Entidades de documentação, provas e notas do processo.

conteúdo textual da nota (*content*), a data do registo (*creation date*) e o autor (*author*).

No que respeita às relações entre estas entidades e o módulo central do sistema, cada entidade *Report* encontra-se vinculada a um único processo específico, estabelecendo-se uma relação de 1-para-1 ou de *N*-para-1 com a entidade *Process*, dependendo da permissão de múltiplos relatórios por sinistro.

Relativamente à relação com a entidade *Prove*, esta possui uma dependência direta com o processo ou com o relatório, garantindo que um *Process* possa conter múltiplos elementos de prova associados, caracterizando uma relação de 1-para-*N*.

Por último, no que concerne à relação com a entidade *Note*, esta apresenta-se associada de forma hierárquica ao *Process*. Um único processo pode agregar múltiplas notas explicativas ao longo do seu ciclo de vida, definindo uma relação de 1-para-*N*, o que viabiliza a criação de uma linha temporal auditável de comunicações e observações internas.

Processos, Área e Utilizador

No que respeita ao conjunto de entidades responsável pela distribuição de tarefas, pela definição do âmbito de especialidade dos utilizadores e pelo registo de auditoria do sistema, destacam-se as entidades *Process*, *User*, *Area* e *Activity*. Estas componentes são essenciais para operacionalizar a alocação de peritos, segmentar as competências técnicas e manter um histórico imutável de todas as ações realizadas na plataforma. A estrutura destas interligações encontra-se representada na Figura 2.6.

Relativamente à entidade *Area*, esta destina-se a classificar o ramo ou a tipologia de sinistro sob a qual o processo se enquadra. É composta por um identificador único (*id*), um nome descritivo da área de atuação (*name*) e um código de classificação interna (*area code*).

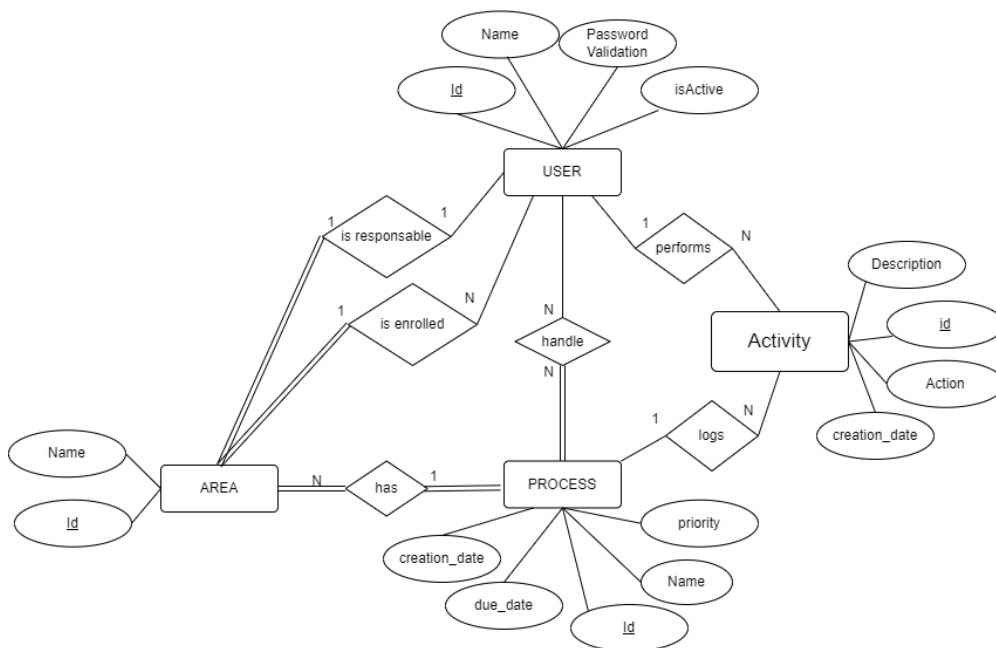


Figura 2.6: Mapeamento relacional incluindo processos, utilizadores, áreas de especialidade e atividades.

O seu propósito é segmentar os processos de acordo com a natureza do incidente para garantir que são tratados por profissionais qualificados na matéria.

No que concerne à entidade *Activity*, esta funciona como um registo detalhado de auditoria das operações efetuadas no sistema. É composta por um identificador único (*id*), a ação específica realizada sobre um determinado sinistro (*action*), a data e hora do registo (*creation date*) e uma descrição textual detalhada do evento (*description*).

No que concerne às relações entre estas entidades, a associação entre *User* e *Process* materializa a atribuição de responsabilidades dentro da aplicação. Um utilizador, consoante o seu papel, pode estar associado a múltiplos processos em simultâneo, e um processo pode envolver a participação de diferentes utilizadores ao longo do seu ciclo de vida, caracterizando uma relação de muitos-para-muitos (*N-para-N*). Nesta associação, são guardados dados como a data de atribuição (*assignment date*) do utilizador ao respetivo caso.

Relativamente à ligação entre as entidades *User* e *Area*, esta desdobra-se em dois relacionamentos distintos com propósitos e cardinalidades diferentes:

- **Afetação à Área (*N-para-1*):** Determina a área de especialidade técnica e competência profissional à qual cada colaborador está afetado. Sob esta regra, cada utilizador pode estar estritamente associado a apenas uma única área de especialidade (por exemplo, pertencer exclusivamente ao ramo automóvel ou ao ramo de incêndios), enquanto uma mesma área de atuação pode contar com o suporte de múltiplos utilizadores aptos, caracterizando uma relação de muitos-para-um (*N-para-1*) da perspectiva

do utilizador.

- **Responsável pela Área (1-para-1):** Define de forma unívoca o utilizador que assume a chefia ou a responsabilidade máxima por uma determinada área de atuação. Trata-se de uma relação de um-para-um (1-para-1), garantindo que cada área possui apenas um responsável encarregue da sua coordenação.

Por último, no que concerne às relações associadas à entidade *Activity*, esta interliga-se de forma fulcral com os utilizadores e os processos:

- **Relação com *User* (1-para-*N*):** Um utilizador pode realizar múltiplas ações ao longo do tempo, gerando várias atividades no sistema, ao passo que cada atividade está associada a apenas um utilizador autor. Esta ligação caracteriza uma relação de 1-para-*N*, permitindo rastrear o autor de cada modificação.
- **Relação com *Process* (1-para-*N*):** Cada processo centraliza o seu próprio histórico operacional, acumulando múltiplas atividades ao longo do seu ciclo de vida, enquanto cada atividade individual pertence a um único processo. Define-se, assim, uma relação de 1-para-*N*, o que viabiliza a consulta de uma linha temporal completa de eventos por cada sinistro.

2.2.3 Modelo completo

A abordagem descrita permitiu deslocar determinadas regras críticas de consistência para a própria base de dados, reduzindo a dependência exclusiva da camada aplicacional para a validação de estados e garantindo um comportamento mais robusto e consistente do sistema.

Como síntese de toda a estrutura relacional planeada, a Figura 2.7 demonstra, de forma completa e integrada, o modelo Entidade-Associação global desenvolvido para suportar o ecossistema da aplicação.

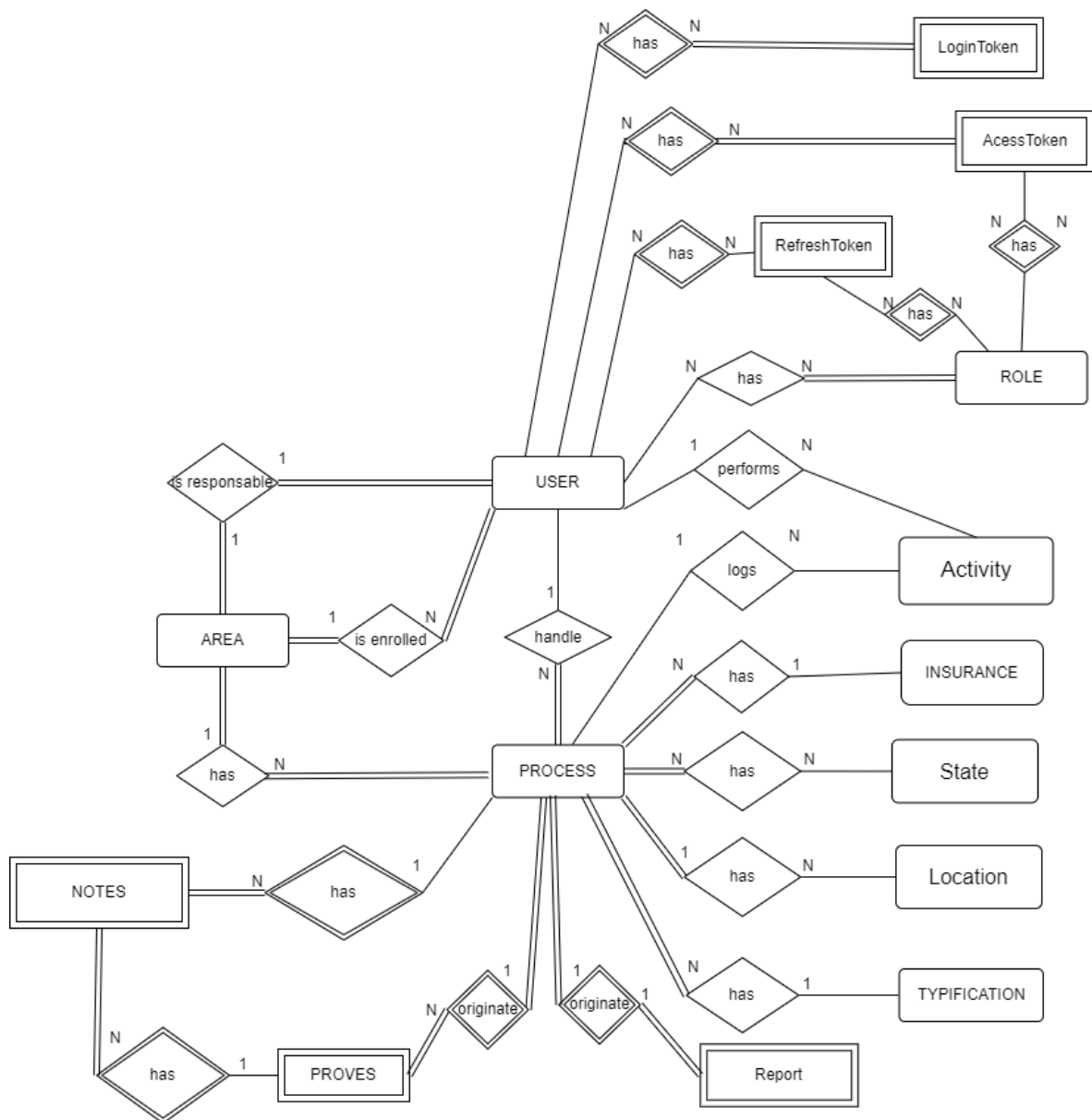


Figura 2.7: Modelo entidade-associação completo (sem atributos).

2.2.4 Implementação

A componente de persistência de dados da aplicação foi implementada recorrendo ao *PostgreSQL*, sendo responsável pelo armazenamento e gestão da informação associada aos diferentes módulos do sistema.

A organização da base de dados encontra-se dividida em diferentes ficheiros *.sql*, cada um com responsabilidades específicas no processo de configuração e inicialização do sistema. Esta separação permite uma melhor modularização da infraestrutura de dados, facilitando a manutenção, evolução e reutilização dos *scripts*.

Existe um ficheiro dedicado à criação da estrutura da base de dados, responsável pela definição das tabelas, relações, restrições de integridade e restantes objetos necessários ao modelo relacional. Para além deste, existem ficheiros específicos para o preenchimento inicial de tabelas contendo informação estática ou previamente conhecida, nomeadamente tabelas como *State*, *Role* e *Area*. Este mecanismo permite garantir que determinados dados essenciais ao funcionamento do sistema se encontram disponíveis desde o momento da inicialização da aplicação.

Adicionalmente, a base de dados integra um conjunto de *triggers* (gatilhos) responsáveis por automatizar determinadas regras de negócio diretamente ao nível da persistência de dados. A opção pela utilização de *triggers* fundamenta-se na sua natureza reativa e orientada a eventos. Ao contrário das funções ou procedimentos armazenados tradicionais, que exigem uma invocação explícita através de código na camada aplicacional, os *triggers* reagem de forma automática e imediata aquando da ocorrência de um evento específico na base de dados (como a inserção, modificação ou remoção de registos). Esta execução transparente garante que as regras críticas de integridade e consistência da informação são salvaguardadas de forma autónoma diretamente no motor do *PostgreSQL*, independentemente de qual o serviço ou operação que originou o pedido.

Um dos *triggers* implementados é responsável por fechar automaticamente o estado anterior de um processo quando um novo estado é inserido na tabela de estados do processo. Desta forma, sempre que ocorre uma mudança de estado, o estado anteriormente ativo passa automaticamente a possuir uma data de fim associada.

Para além do *trigger* supracitado, também existe um *trigger* responsável por garantir que cada processo possui apenas um estado ativo em simultâneo, assegurando que apenas um registo apresenta o campo *end_date* com valor nulo para um determinado processo.

Foi igualmente implementado um *trigger* responsável por atualizar automaticamente o campo *updated_at* sempre que um relatório é modificado. Esta abordagem permite manter informação temporal consistente relativamente às alterações efetuadas sobre os relatórios armazenados no sistema.

Por fim, foi implementado um mecanismo automático associado à criação e atribuição de

processos. Após a inserção de um novo processo, é criado automaticamente o estado inicial correspondente. Caso o processo possua simultaneamente um investigador e um supervisor associados, o estado definido é *assigned*; caso contrário, o sistema atribui automaticamente o estado *not_assigned*. Adicionalmente, quando um processo passa posteriormente a possuir investigador e supervisor definidos, um *trigger* deteta essa alteração e insere automaticamente o estado *assigned*.

Esta abordagem permite deslocar determinadas regras críticas de consistência para a própria base de dados, reduzindo a dependência exclusiva da camada aplicacional para validação de estados e garantindo um comportamento mais robusto e consistente do sistema.

2.3 Backend e segurança

A componente *backend* da aplicação foi implementada na linguagem Kotlin, utilizando a framework *Spring Boot* [4] para a construção de uma *API REST*. Esta *API* suporta as operações principais do sistema, incluindo a autenticação e autorização de utilizadores, a gestão de utilizadores, a criação e acompanhamento de processos, a gestão de áreas, a consulta de histórico e as restantes funcionalidades disponibilizadas à interface *web*.

A segurança da *API* é assegurada através do *Spring Security* [5]. Este componente permite integrar mecanismos de autenticação e autorização diretamente na camada *HTTP*, garantindo que os *endpoints* da aplicação apenas são executados por utilizadores autenticados e, quando aplicável, com o papel adequado. Para este efeito, a aplicação utiliza *JSON Web Tokens (JWT)* [6], que transportam a identificação do utilizador e o papel ativo na sessão.

O processo de autenticação encontra-se dividido em diferentes momentos. Inicialmente, o utilizador efetua *login* com as suas credenciais, recebendo um *token* temporário associado à fase de autenticação inicial. De seguida, seleciona o papel que pretende desempenhar na sessão, de entre os papéis que lhe estão atribuídos. Após esta seleção, são gerados dois *tokens*: um *access token* e um *refresh token*. O *access token* é utilizado nos pedidos à *API* protegida, enquanto o *refresh token* permite renovar o acesso sem obrigar o utilizador a repetir o processo de *login*.

O controlo de acessos segue uma abordagem baseada em papéis, também conhecida como *RBAC (Role-Based Access Control)* [7]. No sistema foram considerados papéis alinhados com o funcionamento do processo de averiguação, distinguindo responsabilidades de administração, triagem, averiguação, supervisão e gestão. Esta separação permite que cada utilizador aceda apenas às operações compatíveis com a sua função no fluxo de trabalho, reduzindo a exposição de funcionalidades sensíveis e tornando mais claro o percurso de cada processo.

Ao nível da camada *HTTP*, as permissões são expressas através de anotações como `@RolesAllowed`, associadas aos métodos dos controladores. Desta forma, cada *endpoint* declara explicitamente os papéis autorizados a executá-lo. Esta abordagem torna a política de autorização mais visível no código e reduz o risco de expor operações sensíveis a utilizadores sem permissão.

A validação dos *access tokens* é realizada por um filtro *HTTP* dedicado, integrado na cadeia de filtros do *Spring Security*. Este filtro extrai o *token* do cabeçalho `Authorization`, valida a sua assinatura, confirma o seu tipo e validade temporal, e cria uma representação do utilizador autenticado no contexto de segurança do Spring. Essa informação fica disponível através do `SecurityContextHolder`, permitindo que os controladores acessem ao identificador do utilizador e ao papel ativo no pedido.

Nem todos os *endpoints* passam pelo mesmo fluxo de autenticação. *Endpoints* como *login*, seleção de papel e renovação de *token* têm um tratamento específico, uma vez que ainda não

utilizam, ou não utilizam exclusivamente, um *access token*. Por esse motivo, o filtro *JWT* é configurado para ignorar estes *endpoints*, permitindo que sejam tratados pelos respetivos mecanismos de autenticação ou renovação.

Esta organização permite separar responsabilidades entre autenticação, autorização e lógica de negócio. O filtro é responsável por identificar o utilizador a partir do *token*, o *Spring Security* aplica as regras gerais de acesso e os controladores delegam nos serviços a execução das operações de domínio. Como resultado, a aplicação consegue controlar o acesso às suas funcionalidades de forma consistente com os diferentes papéis existentes no *workflow*.

2.4 Servidor Frontend

A componente de *frontend* da aplicação foi desenvolvida com recurso ao servidor *Vite*, utilizando *React* e *TypeScript* como principais tecnologias.

A arquitetura do *frontend* encontra-se organizada em diferentes camadas funcionais, nomeadamente páginas, autenticação, comunicação, componentes e estilos (figura 2.8). A camada de páginas é responsável pela organização das diferentes vistas da aplicação, incluindo páginas como autenticação, *dashboard*, gestão de processos e detalhe de processos. A camada de autenticação centraliza a lógica relacionada com o processo de autenticação, armazenamento de tokens de acesso e controlo de sessões dos utilizadores.

A camada de comunicação é responsável pela interação com a *API backend*, assegurando o envio e receção de dados entre o cliente e o servidor. Já a camada de componentes contém elementos reutilizáveis da interface gráfica, como tabelas, formulários, botões e janelas de *pop-up*, promovendo maior reutilização de código e consistência visual entre as diferentes páginas da aplicação. Por fim, a camada de estilos concentra os estilos globais como por exemplo tipografia utilizada, utilizando *CSS* para definição da identidade visual da plataforma.

De forma a garantir controlo de acesso às diferentes áreas da aplicação, foram implementados mecanismos de proteção de rotas no *frontend*. As rotas que requerem autenticação encontram-se encapsuladas pelo componente **AuthenticatedRoute**, responsável por verificar a existência de um *token* de autenticação válido antes de permitir o acesso à página pretendida. Caso o utilizador não esteja autenticado, este é redirecionado para a página de autenticação.

Adicionalmente, determinadas páginas da plataforma requerem permissões específicas associadas ao perfil do utilizador autenticado. Para este efeito, foi desenvolvido o componente **ProtectedRoute**, responsável por validar se o utilizador possui o papel necessário para aceder à rota em questão. Caso o utilizador não possua as permissões adequadas, o sistema efetua o redirecionamento para uma página de acesso negado.

Apesar da existência destes mecanismos de proteção no *frontend*, toda a validação de permissões e autorização é igualmente garantida no *backend* da aplicação, assegurando que

utilizadores sem permissões adequadas não conseguem aceder a recursos protegidos através de manipulação do cliente.

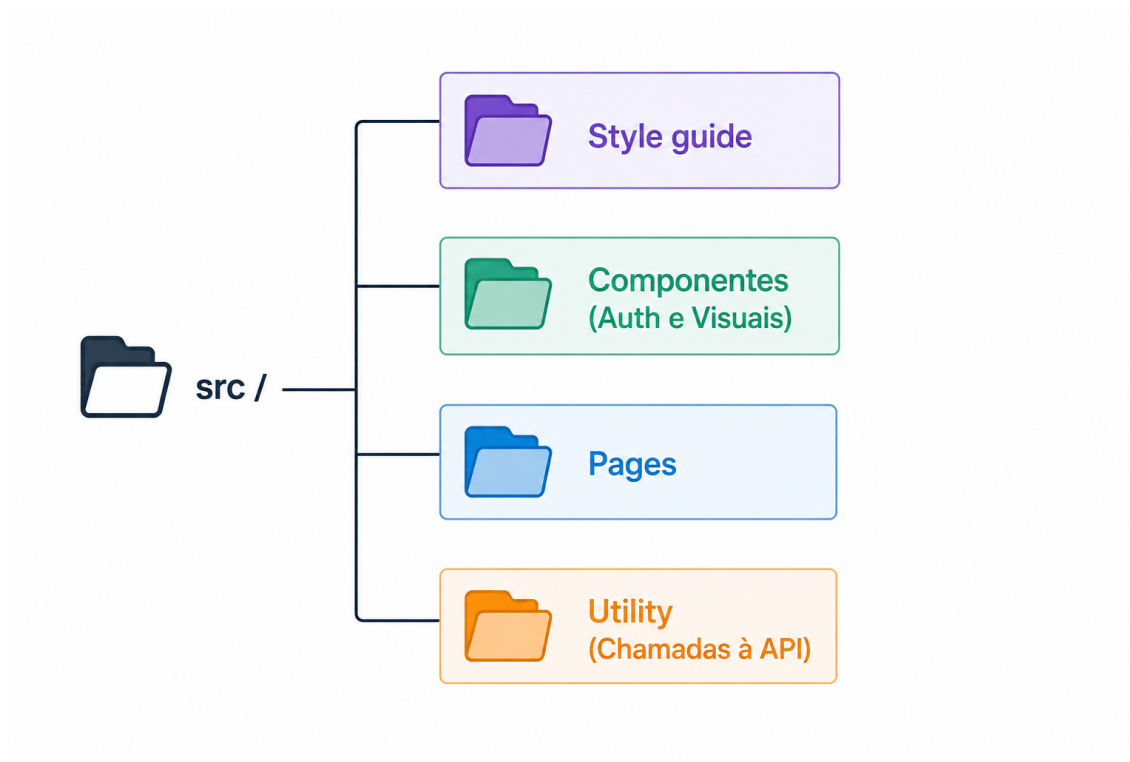


Figura 2.8: Estrutura de pastas da componente *frontend*

2.4.1 Bibliotecas Utilizadas

Para o desenvolvimento da componente *frontend* foram utilizadas diversas bibliotecas externas ao projeto, descritas de seguida.

- **Material UI** [8] (v7.3.9) — biblioteca de componentes visuais. Foi utilizada para componentes como `Modal`, `DropDownMenu` e `LoadingComponent`.
- **react-i18next** [9] (v16.6.1) — biblioteca de internacionalização que permite a gestão de traduções na aplicação, suportando múltiplos idiomas de forma dinâmica.
- **jose** [10] (v6.2.3) — biblioteca utilizada para a decodificação de *JSON Web Tokens* (JWT) no lado do cliente.
- **notistack** [11] (v3.0.2) — biblioteca para apresentação de notificações do tipo *snackbar* ao utilizador.

Capítulo 3

Demonstração da Aplicação

Este capítulo apresenta os principais fluxos da aplicação desenvolvida, com o objetivo de demonstrar o funcionamento das funcionalidades implementadas até à fase atual do projeto. A demonstração encontra-se organizada de acordo com o percurso natural de utilização da plataforma, desde a autenticação até à gestão e acompanhamento dos processos de averiguação.

3.1 Autenticação e seleção de papel

O primeiro contacto do utilizador com a aplicação ocorre através da página de autenticação, representada na Figura 3.1. Nesta página, o utilizador introduz as suas credenciais para iniciar sessão. Após uma autenticação bem sucedida, a aplicação apresenta os papéis disponíveis para esse utilizador.

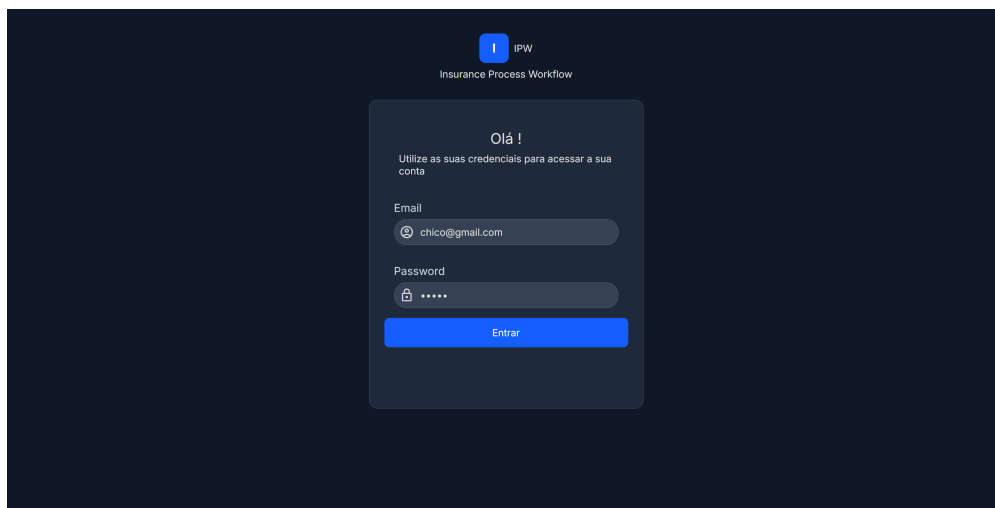


Figura 3.1: Página de autenticação da aplicação.

A seleção de papel, apresentada na Figura 3.2, permite que o mesmo utilizador desempenhe diferentes funções no sistema, quando aplicável. Esta decisão define o contexto da sessão e condiciona as funcionalidades disponíveis na interface, bem como as rotas que podem ser acedidas no backend.

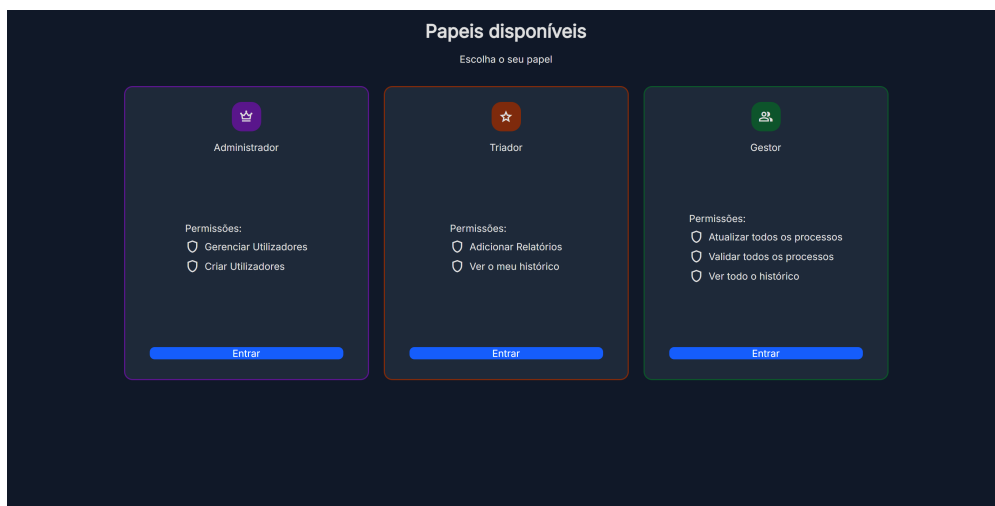


Figura 3.2: Seleção do papel ativo na sessão do utilizador.

3.2 Listagem e acompanhamento de processos

Depois da seleção do papel, o utilizador é encaminhado para a área principal da aplicação, cujo conteúdo varia consoante o papel selecionado. Para os papéis diretamente envolvidos no acompanhamento de processos, esta área pode apresentar uma visão geral dos processos relevantes, como exemplificado na Figura 3.3, incluindo informação resumida como o nome do processo, a localização, a área associada, a data limite e a prioridade.

The screenshot shows the 'Averiguador Dashboard' with a sidebar on the left and a main content area. The main content area features a table titled 'Processos Recentes' (Recent Processes). The table has the following columns: Nome, Localização, Data de Criação, Expira em, Prioridade, and Estado. The data rows are as follows:

Nome	Localização	Data de Criação	Expira em	Prioridade	Estado
Process Beta	Rua Augusta 1, Lisbon	31/05/2026	20/06/2026	Medio	Atribuido
Process Epsilon	Rua Augusta 1, Lisbon	31/05/2026	10/06/2026	Alta	Atribuido
Process Delta	Rua Augusta 1, Lisbon	31/05/2026	15/07/2026	Baixa	Atribuido
Process Alpha	Rua Augusta 1, Lisbon	31/05/2026	30/06/2026	Baixa	Atribuido
Process Gamma	Rua Augusta 1, Lisbon	31/05/2026	15/06/2026	Alta	Atribuido

At the bottom right of the table, there are navigation controls: a left arrow, '1/1', and a right arrow.

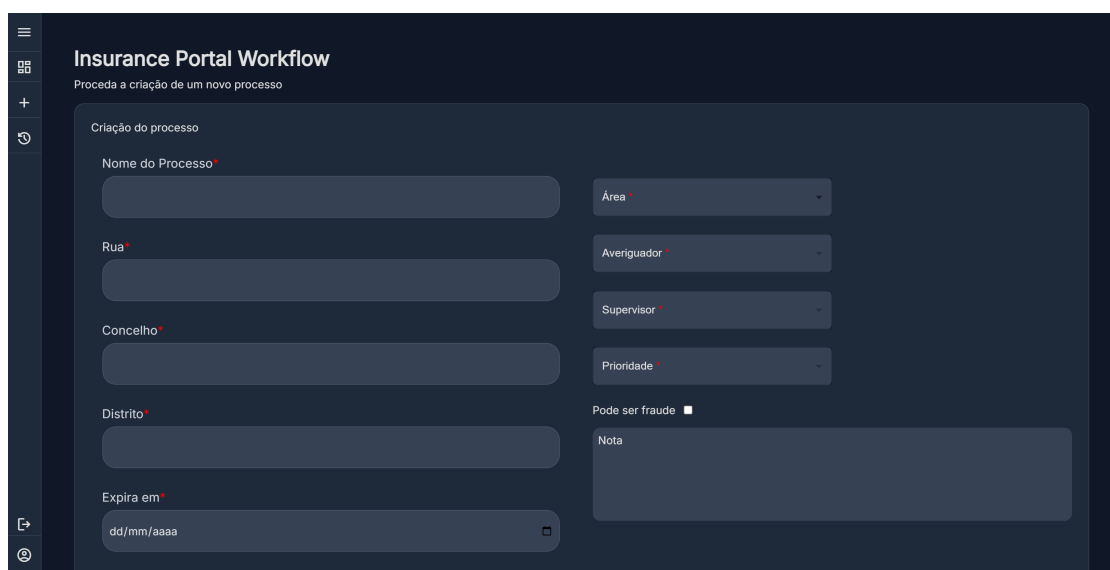
Figura 3.3: Exemplo de listagem de processos disponível para papéis envolvidos no acompanhamento.

Quando disponível, a tabela de processos serve como ponto de entrada para a consulta detalhada de cada processo. Esta abordagem permite que os utilizadores envolvidos no *workflow* acompanhem o estado dos processos de forma centralizada, reduzindo a dependência

de informação dispersa por diferentes ficheiros ou canais de comunicação.

3.3 Criação de processos

A criação de processos é uma funcionalidade associada ao papel de triador. Neste fluxo, representado na Figura 3.4, o triador preenche a informação inicial do processo, incluindo dados como o nome, localização, área, prioridade, data limite, seguradora e tipificação. Quando aplicável, o triador pode também atribuir manualmente um averiguador e um supervisor ao processo.



The screenshot shows a web interface titled "Insurance Portal Workflow" with the subtitle "Proceda a criação de um novo processo". The form is titled "Criação do processo" and contains the following fields:

- Nome do Processo* (text input)
- Área* (dropdown menu)
- Rua* (text input)
- Averiguador* (dropdown menu)
- Concelho* (text input)
- Supervisor* (dropdown menu)
- Distrito* (text input)
- Prioridade* (dropdown menu)
- Expira em* (date input, format dd/mm/aaaa)
- Pode ser fraude (checkbox)
- Nota (text area)

Figura 3.4: Formulário de criação de um novo processo pelo triador.

Este fluxo representa uma das funcionalidades centrais da aplicação, uma vez que formaliza a entrada de novos processos no sistema e permite associar, desde o início, os intervenientes responsáveis pelo seu acompanhamento.

3.4 Detalhe do processo

A página de detalhe do processo, ilustrada na Figura 3.5, disponibiliza informação mais completa sobre um processo específico. Nesta área são apresentados dados como os intervenientes atribuídos, a área, datas relevantes, prioridade, estado atual e informação associada ao relatório de averiguação.

Esta página permite concentrar num único local a informação operacional do processo, facilitando o acompanhamento por parte dos diferentes papéis envolvidos no *workflow*.

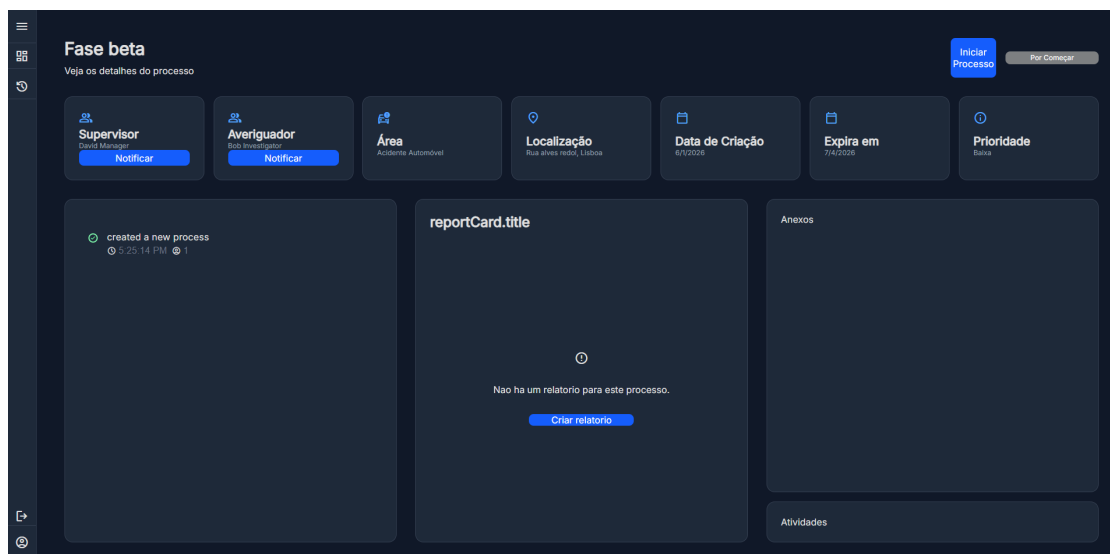


Figura 3.5: Página de detalhe de um processo de averiguação.

3.5 Gestão de utilizadores

A aplicação inclui também funcionalidades de administração, acessíveis a utilizadores com o papel de administrador. Através desta área, exemplificada na Figura 3.6, é possível consultar utilizadores, criar novos utilizadores, alterar papéis e gerir a associação de utilizadores a áreas.

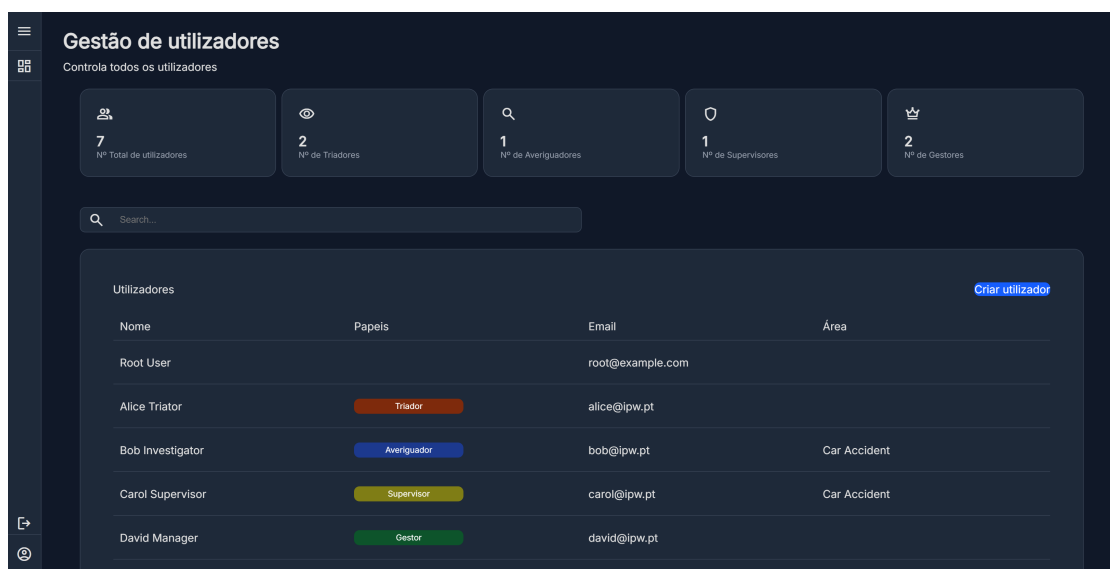


Figura 3.6: Interface de gestão de utilizadores disponível ao administrador.

Esta funcionalidade é relevante para manter o sistema alinhado com a organização interna da CAPT, permitindo refletir alterações nas equipas, responsabilidades e áreas de especialização.

3.6 Perfil e histórico de atividade

Por fim, a aplicação disponibiliza uma área de perfil, onde o utilizador pode consultar informação associada à sua conta e à sua atividade recente, como apresentado na Figura 3.7. A existência de histórico e registo de interações contribui para a rastreabilidade dos processos, permitindo perceber que operações foram realizadas ao longo do tempo.

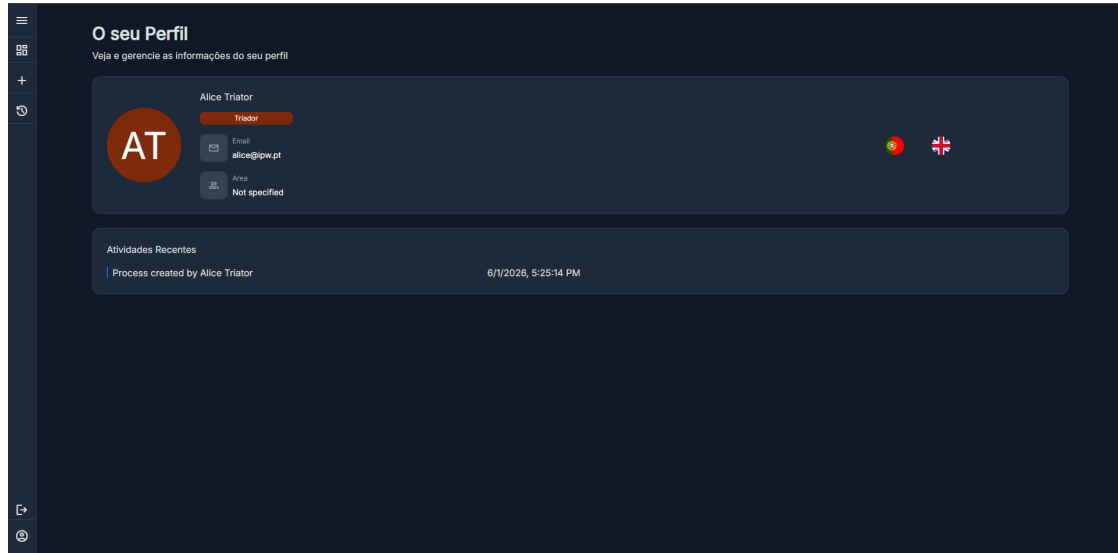


Figura 3.7: Página de perfil e consulta de atividade recente do utilizador.

Em conjunto, estes fluxos demonstram a base funcional da plataforma *IPW* e a forma como a aplicação suporta a gestão estruturada dos processos de averiguação.

Capítulo 4

Conclusão e Trabalho Futuro

4.1 Conclusão

O projeto *IPW* teve como objetivo principal conceber e desenvolver uma plataforma *web* para apoiar a gestão de processos de averiguação no contexto da CAPT Consulting. A solução proposta procura responder às limitações identificadas no processo atual, nomeadamente a dispersão da informação, a dificuldade em acompanhar o estado dos processos e a inexistência de um *workflow* centralizado e estruturado.

Ao longo do desenvolvimento foi definida uma arquitetura composta por uma aplicação *frontend*, uma *API backend* e uma base de dados relacional. O *frontend* disponibiliza a interface de utilização da plataforma, enquanto o *backend* concentra a lógica de negócio, a autenticação, a autorização e o acesso aos dados. A base de dados foi modelada de forma a representar os principais conceitos do domínio, incluindo utilizadores, papéis, áreas, processos, estados e histórico.

Na fase atual, a aplicação já inclui funcionalidades fundamentais, como autenticação de utilizadores, seleção de papel, proteção de rotas, gestão de utilizadores, criação e listagem de processos, associação de intervenientes, consulta de informação relevante para o acompanhamento dos processos e suporte inicial à anexação de ficheiros. Estas funcionalidades permitem validar a estrutura base da solução e demonstrar a viabilidade da abordagem adotada.

O desenvolvimento permitiu também consolidar decisões técnicas importantes, como a utilização de *JWT* para autenticação, a aplicação de controlo de acessos baseado em papéis e a separação do *backend* em camadas com responsabilidades distintas. Apesar de ainda existirem funcionalidades por concluir, a versão atual estabelece uma base consistente para a evolução do sistema.

4.2 Trabalho futuro

Como trabalho futuro, deverá ser consolidada a integração da anexação de ficheiros na interface da aplicação. Esta evolução deverá incidir sobretudo na experiência de utilização

associada ao carregamento, visualização e remoção de fotografias, vídeos e documentos relevantes para cada processo de averiguação.

Outra melhoria prevista consiste no aprofundamento da integração entre o *frontend* e o *backend*. Embora a aplicação já possua vários componentes e *endpoints* implementados, ainda existem áreas que necessitam de maior integração, validação e tratamento de estados de erro. Esta evolução permitirá aproximar a experiência de utilização do fluxo final pretendido.

Está também prevista a melhoria dos mecanismos de monitorização e visualização de indicadores operacionais. Esta evolução poderá incluir informação agregada sobre prazos, distribuição de trabalho por utilizador ou área, estados dos processos e outros indicadores relevantes para a gestão.

Adicionalmente, poderá ser estudada a atribuição automática de processos a averiguadores e supervisores, tendo em conta critérios como área de especialização, disponibilidade ou carga de trabalho. Esta funcionalidade foi considerada como requisito opcional, uma vez que depende da estabilização prévia do fluxo principal de criação e acompanhamento de processos.

Referências

- [1] e-Clic. e-clic. <https://e-clic.net>, 2026. Acedido em maio de 2026.
- [2] DocuWare. Docuware. <https://start.docuware.com>, 2026. Acedido em maio de 2026.
- [3] Microsoft. Sharepoint. <https://www.microsoft.com/pt-pt/microsoft-365/sharepoint/collaboration>, 2026. Acedido em maio de 2026.
- [4] Equipa Spring. Spring boot reference documentation. <https://docs.spring.io/spring-boot/>, 2026. Acedido em maio de 2026.
- [5] Equipa Spring. Spring security reference. <https://docs.spring.io/spring-security/reference/>, 2026. Acedido em maio de 2026.
- [6] M. Jones, J. Bradley, and N. Sakimura. RFC 7519: Json web token (jwt). <https://www.rfc-editor.org/rfc/rfc7519>, 2015. Acedido em maio de 2026.
- [7] D. F. Ferraiolo, D. R. Kuhn, and R. Chandramouli. Role-based access control. <https://csrc.nist.gov/projects/role-based-access-control>, 2007. Acedido em maio de 2026.
- [8] Equipa Material UI. Material ui: React component library. <https://mui.com/material-ui/getting-started/>, 2026. Acedido em maio de 2026.
- [9] Equipa i18next. react-i18next: Internationalization for react. <https://react.i18next.com/latest/usetranslation-hook>, 2026. Acedido em maio de 2026.
- [10] Equipa Okta. jose: Javascript object signing and encryption. <https://www.npmjs.com/package/jose>, 2026. Acedido em maio de 2026.
- [11] Equipa notistack. notistack: SnackBar notification library for react. <https://notistack.com/api-reference>, 2026. Acedido em maio de 2026.